Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №1

Численные методы нулевого порядка для поиска безусловного экстремума
тема

Преподаватель _____ В.В. Тынченко
подпись, дата — инициалы, фамилия

Студент КИ24-17/1Б, 032428518 _____ Э. А. Алиев
номер группы, зачетной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026

1. Постановка задачи

Требуется программно реализовать численные методы безусловной оптимизации варианта 1, проверить правильность работы алгоритмов на примерах из учебника А. В. Пантелеева и Т. А. Летовой и провести вычислительные эксперименты. Точность оценивается абсолютной погрешностью найденного минимального значения функции, скорость — количеством вычислений целевой функции.

Для варианта 1 заданы методы (метод равномерного поиска, метод квадратичной интерполяции) и функции, которые изображены на рисунках 1 и 2.

$$f(x) = (2x + 3)^2 - 8x - 10 \rightarrow min$$

Интервал неопределённости [-6,6].

Рисунок 1 – Первая функция

$$f(x) = 4(x - 5)^2 + e^x(x - 6)^2 \rightarrow min$$

Интервал неопределённости [0,6].

Рисунок 2 – Вторая функция

Для каждого исследуемого параметра необходимо построить графики зависимости погрешности и количества вычислений функции, дать краткий комментарий и представить итоговые результаты в сводной таблице, построенной программно.

2. Краткое описание методов

2.1. Метод равномерного поиска

Отрезок [a; b] разбивается на n равных частей. Значение функции вычисляется в n + 1 узле, после чего выбирается точка с наименьшим значением. При увеличении n шаг сетки уменьшается, поэтому точность обычно возрастает, но число вычислений функции растёт линейно и равно n + 1.

2.2. Метод квадратичной интерполяции

Метод предполагает, что на отрезке [a; b] выбраны три точки $x1 < x2 < x3$, для которых значение в средней точке не превышает значений на концах. Через эти три точки и соответствующие значения функции строится интерполирующая парабола, и вычисляется абсцисса её вершины — точка $\bar{x}$, дающая уточнённое приближение к минимуму. Далее одна из крайних точек заменяется точкой $\bar{x}$ так, чтобы минимум снова оказался внутри новой тройки точек, и процесс повторяется. Итерации останавливаются, когда очередное смещение вершины параболы становится меньше заданной точности eps. Метод сходится очень быстро (сверхлинейно) на гладких унимодальных функциях, однако если функция на заданном отрезке не является строго унимодальной, метод может сойтись не к глобальному, а к ближайшему локальному экстремуму.

2.3. Определение точных минимумов функций

Первая функция раскрывается как $f1(x) = (2x + 3)^2 - 8x - 10 = 4x^2 + 4x - 1$. Из условия $f1'(x) = 8x + 4 = 0$ получаем $x^* = -0{,}5$, что лежит внутри отрезка $[-6; 6]$, и $f1(x^*) = -2$.

Вторая функция $f2(x) = 4(x - 5)^2 + e^x(x - 6)^2$ на отрезке [0; 6] не является строго унимодальной: производная $f2'(x) = 8(x - 5) + e^x(x - 6)(x - 4)$ обращается в ноль в трёх точках отрезка — приближённо при $x \approx 0{,}668$ (локальный минимум, $f2 \approx 130{,}513$), $x \approx 3{,}917$ (локальный максимум, $f2 \approx 222{,}718$) и $x \approx 5{,}990$ (второй

локальный минимум). Сравнение значений функции в этих точках и на границах отрезка (f2(0) = 136, f2(6) = 4) показывает, что глобальный минимум достигается вблизи правой границы: x* ≈ 5,990035845, f2(x*) ≈ 3,960340953. Корень уравнения f2′(x) = 0 в окрестности x* найден численно (методом деления пополам/Брента), так как уравнение является трансцендентным и аналитического решения не имеет.

3. Текст программы с комментариями

Программа написана на Python. Для таблиц используется библиотека pandas, для построения графиков — matplotlib. Полный текст программы приведён ниже.

```
"""
Практическая работа №1. Вариант 1.
Методы: равномерный поиск, квадратичная интерполяция (метод парабол).
"""
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import pandas as pd
import matplotlib.pyplot as plt

OUT_DIR = Path("results")
OUT_DIR.mkdir(exist_ok=True)



@dataclass
class Result1D:
    x: float
    fx: float
    evaluations: int



class Counter:
    """Считает обращения к целевой функции."""

    def __init__(self, function: Callable):
        self.function = function
        self.count = 0

    def __call__(self, x):
        self.count += 1
        return self.function(x)



def uniform_search(function: Callable[[float], float], a: float, b: float, n: int) -> Result1D:
    """Равномерно проверяет n + 1 точку отрезка [a, b]."""
    if n < 1:
        raise ValueError("Количество частей n должно быть положительным")
    f = Counter(function)
    best_x = a
    best_value = f(a)
    step = (b - a) / n
```

```
    for i in range(1, n + 1):
        x = a + i * step
        value = f(x)
        if value < best_value:
            best_x, best_value = x, value
    return Result1D(best_x, best_value, f.count)



def quadratic_interpolation(function: Callable[[float], float], a: float, b: float,
                            eps: float, max_iter: int = 500) -> Result1D:
    """Метод квадратичной интерполяции (метод парабол).

    Через три точки x1 < x2 < x3 (f(x2) не больше значений на краях)
    строится парабола, вычисляется абсцисса её вершины x, после чего
    тройка точек сужается так, чтобы минимум снова оказался внутри
    интервала. Итерации останавливаются, когда очередная вершина
    параболы смещается меньше, чем на eps.
    """
    f = Counter(function)
    x1, x3 = a, b
    x2 = (a + b) / 2
    f1, f2, f3 = f(x1), f(x2), f(x3)

    for _ in range(max_iter):
        denom = (x2 - x1) * (f2 - f3) - (x2 - x3) * (f2 - f1)
        if abs(denom) < 1e-14:
            break
        x_bar = x2 - 0.5 * ((x2 - x1) ** 2 * (f2 - f3) - (x2 - x3) ** 2 * (f2 - f1)) / denom
        x_bar = min(max(x_bar, x1), x3)
        f_bar = f(x_bar)

        if abs(x_bar - x2) < eps:
            if f_bar < f2:
                x2, f2 = x_bar, f_bar
            break

        if x_bar < x2:
            if f_bar < f2:
                x3, f3 = x2, f2
                x2, f2 = x_bar, f_bar
            else:
                x1, f1 = x_bar, f_bar
        else:
            if f_bar < f2:
                x1, f1 = x2, f2
                x2, f2 = x_bar, f_bar
            else:
                x3, f3 = x_bar, f_bar

    return Result1D(x2, f2, f.count)



def f1(x: float) -> float:
    return (2 * x + 3) ** 2 - 8 * x - 10



def f2(x: float) -> float:
    return 4 * (x - 5) ** 2 + math.exp(x) * (x - 6) ** 2



FUNCTIONS_1D = {
    "f1": (f1, -6.0, 6.0, -0.5, -2.0),
    "f2": (f2, 0.0, 6.0, 5.990035844581383, 3.9603409528734455),
}



def run_control_test() -> list[str]:
    """
    Проверка методов на функции f(x) = 2x^2 - 12x из примеров 5.2-5.3
```

```
    учебника Пантелеева и Летовой. Точный минимум на отрезке [0; 10]:
    x* = 3, f* = -18.
    """
    test = lambda x: 2 * x ** 2 - 12 * x
    uniform = uniform_search(test, 0, 10, 10)
    quad = quadratic_interpolation(test, 0, 10, 1e-4)

    assert abs(uniform.x - 3) <= 0.01
    assert abs(uniform.fx - (-18)) <= 0.01
    assert abs(quad.x - 3) <= 1e-4
    assert abs(quad.fx - (-18)) <= 1e-4

    return [
        f"Равномерный поиск: x*={uniform.x:.6f}, f*={uniform.fx:.6f}, "
        f"вычислений={uniform.evaluations}",
        f"Квадратичная интерполяция: x*={quad.x:.6f}, f*={quad.fx:.6f}, "
        f"вычислений={quad.evaluations}",
    ]


def one_dimensional_experiments() -> pd.DataFrame:
    rows: list[dict] = []
    for name, (function, a, b, true_x, true_value) in FUNCTIONS_1D.items():
        for n in range(20, 1001, 20):
            result = uniform_search(function, a, b, n)
            rows.append({
                "method": "Равномерный поиск", "function": name,
                "parameter": "n", "value": n,
                "x": result.x, "fx": result.fx,
                "error": abs(result.fx - true_value),
                "evaluations": result.evaluations,
            })

        eps = 0.005
        while eps <= 0.5 + 1e-9:
            result = quadratic_interpolation(function, a, b, eps)
            rows.append({
                "method": "Квадратичная интерполяция", "function": name,
                "parameter": "eps", "value": eps,
                "x": result.x, "fx": result.fx,
                "error": abs(result.fx - true_value),
                "evaluations": result.evaluations,
            })
            eps += 0.005

    return pd.DataFrame(rows)


def plot_metric(frame: pd.DataFrame, method: str, parameter: str, metric: str, filename: str) -> None:
    subset = frame[(frame["method"] == method) & (frame["parameter"] == parameter)]
    grouped = subset.groupby(["function", "value"], as_index=False)[metric].mean()

    fig, ax = plt.subplots(figsize=(8.2, 4.7))
    for function, data in grouped.groupby("function"):
        ax.plot(data["value"], data[metric], marker="o", markersize=3, linewidth=1.4, label=function)

    ax.set_xlabel({"n": "Количество частей n", "eps": "Точность eps"}[parameter])
    ax.set_ylabel("Погрешность" if metric == "error" else "Количество вычислений функции")
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(OUT_DIR / filename, dpi=180)
    plt.close(fig)


def build_summary(all_results: pd.DataFrame) -> pd.DataFrame:
    descriptions = {
        ("Равномерный поиск", "n"): "n = 20...1000, шаг 20",
        ("Квадратичная интерполяция", "eps"): "eps = 0,005...0,500, шаг 0,005",
    }
```

```
    rows = []
    for (method, function, parameter), group in all_results.groupby(["method", "function", "parameter"]):
        rows.append({
            "Метод": method,
            "Функция": function,
            "Параметры": descriptions[(method, parameter)],
            "Погрешность средняя": group["error"].mean(),
            "Погрешность лучшая": group["error"].min(),
            "Погрешность худшая": group["error"].max(),
            "Вычисления средние": group["evaluations"].mean(),
            "Вычисления лучшие": group["evaluations"].min(),
            "Вычисления худшие": group["evaluations"].max(),
        })
    return pd.DataFrame(rows)


def save_table_image(summary: pd.DataFrame) -> None:
    shown = summary.copy()
    for column in ["Погрешность средняя", "Погрешность лучшая", "Погрешность худшая"]:
        shown[column] = shown[column].map(lambda v: f"{v:.3e}")
    for column in ["Вычисления средние", "Вычисления лучшие", "Вычисления худшие"]:
        shown[column] = shown[column].map(lambda v: f"{v:.1f}")

    fig, ax = plt.subplots(figsize=(16, 3.2))
    ax.axis("off")
    table = ax.table(cellText=shown.values, colLabels=shown.columns, cellLoc="center", loc="center")
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    table.scale(1, 2.0)
    fig.tight_layout()
    fig.savefig(OUT_DIR / "summary_table.png", dpi=200, bbox_inches="tight")
    plt.close(fig)


def main() -> None:
    control_lines = run_control_test()
    all_results = one_dimensional_experiments()
    all_results.to_csv(OUT_DIR / "all_results.csv", index=False)

    plot_metric(all_results, "Равномерный поиск", "n", "error", "uniform_error.png")
    plot_metric(all_results, "Равномерный поиск", "n", "evaluations", "uniform_speed.png")
    plot_metric(all_results, "Квадратичная интерполяция", "eps", "error", "quad_error.png")
    plot_metric(all_results, "Квадратичная интерполяция", "eps", "evaluations", "quad_speed.png")

    summary = build_summary(all_results)
    summary.to_csv(OUT_DIR / "summary.csv", index=False)
    save_table_image(summary)

    lines = ["КОНТРОЛЬНЫЙ ТЕСТ", *control_lines, "", "ТОЧНЫЕ МИНИМУМЫ"]
    for name, (_, _, _, true_x, true_value) in FUNCTIONS_1D.items():
        lines.append(f"{name}: x*={true_x:.9f}, f*={true_value:.9f}")
    lines += ["", "СВОДНАЯ ТАБЛИЦА", summary.to_string(index=False)]
    print("\n".join(lines))


if __name__ == "__main__":
    main()
```

4. Результаты выполненного исследования

4.1. Проверка реализации

Для проверки правильности программной реализации использована функция $f(x) = 2x^2 - 12x$, рассмотренная в примерах 5.2 и 5.3 учебника А. В. Пантелеева и Т. А. Летовой. Точный минимум на отрезке [0; 10] достигается в точке $x^* = 3$, $f(x^*) = -18$.

Равномерный поиск ($n = 10$) точно получил $x^* = 3$, $f^* = -18$ за 11 вычислений функции. Метод квадратичной интерполяции также точно получил $x^* = 3$, $f^* = -18$ всего за 5 вычислений — это ожидаемо, поскольку тестовая функция сама является параболой, и метод квадратичной интерполяции находит минимум точной квадратичной функции за одну итерацию. Следовательно, оба метода реализованы правильно. Результат запуска программы представлен на рисунке 3.

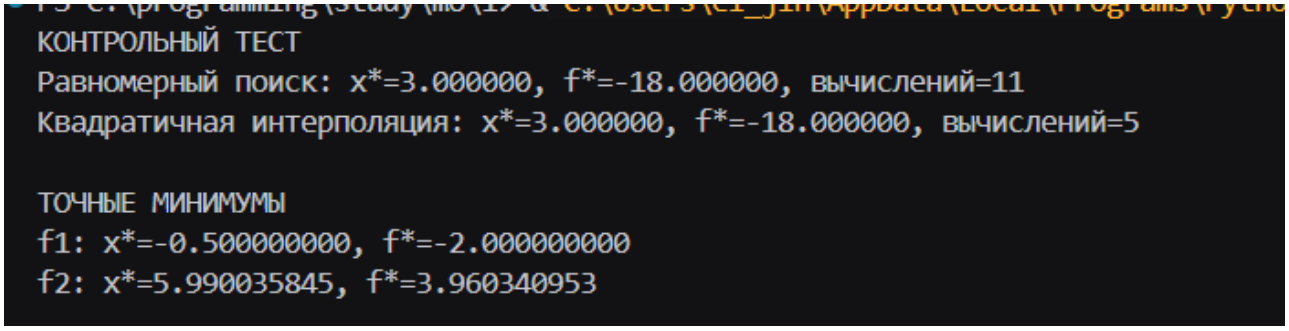


Рисунок 3 – Результат запуска программы

4.2. Метод равномерного поиска

Исследуемый параметр: n от 20 до 1000 с шагом 20. При увеличении n число вычислений возрастает от 21 до 1001 (растёт линейно и точно равно $n + 1$). Зависимость погрешности от n изображена на рисунке 4.

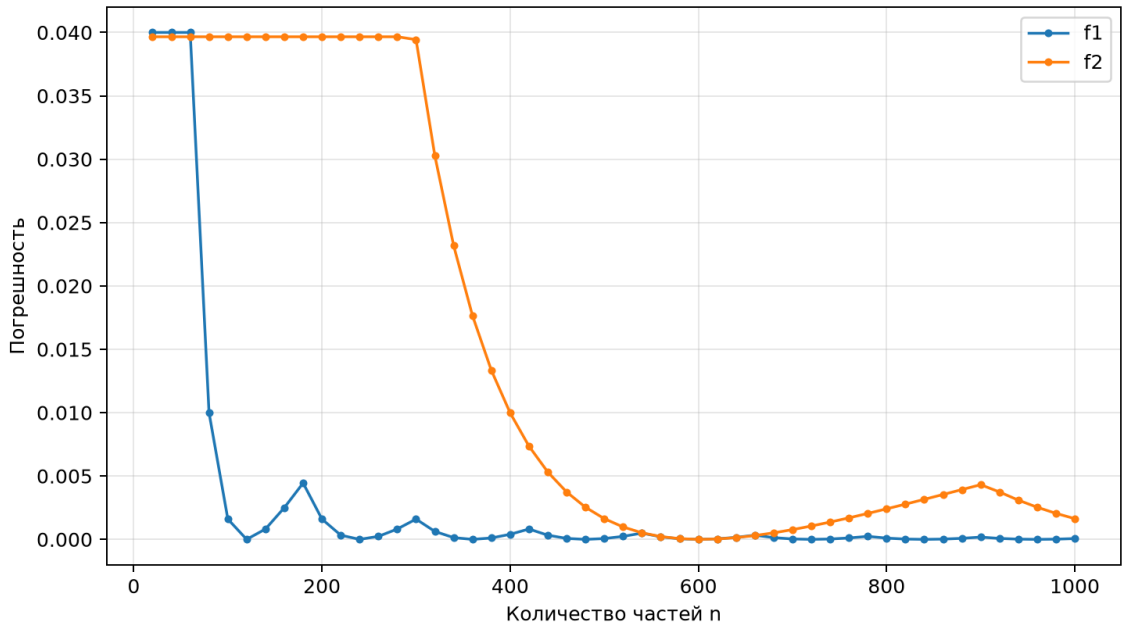


Рисунок 4 – Зависимость погрешности равномерного поиска от n

Для f1 погрешность в среднем составила 0,003 при разбросе от 0 до 0,04 — она убывает при сгущении сетки, а отдельные нулевые значения возникают, когда точка x = −0,5 точно попадает в узел сетки. Для f2 средняя погрешность составила 0,015 (диапазон от $5 \cdot 10^{-7}$ до 0,04): поскольку истинный минимум лежит очень близко к правой границе отрезка ($x^* \approx 5{,}990$), с ростом n сетка всё точнее «нащупывает» эту точку, и погрешность стремится к нулю. Зависимость количества вычислений от параметра n представлена на рисунке 5.

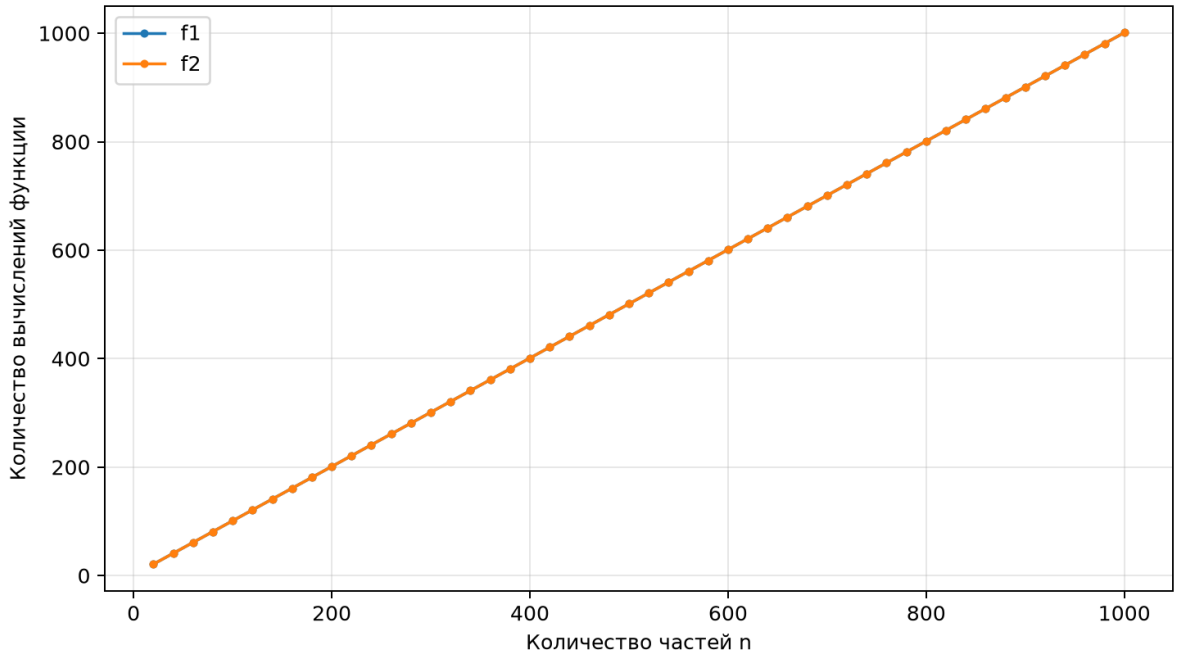


Рисунок 5 – Зависимость числа вычислений равномерного поиска от n

Как видно из рисунка 5, количество вычислений целевой функции увеличивается строго линейно: увеличение числа частей на 20 добавляет ровно 20 вычислений функции (от 21 при n = 20 до 1001 при n = 1000). Это показывает прямой обмен точности на вычислительные.

4.3. Метод квадратичной интерполяции

Для f1 погрешность равна нулю независимо от eps: функция сама является параболой, поэтому вершина построенной параболы совпадает с истинным минимумом уже на первой итерации. Для f2 погрешность практически не зависит от eps и стабильно составляет около 126,55 — метод во всех запусках сходится не к глобальному минимуму на правой границе ($x^* \approx 5{,}990$), а к ближайшему локальному минимуму $x \approx 0{,}668$ ($f \approx 130{,}513$), который находится внутри начальной тройки точек (0; 3; 6). Это прямое следствие того, что отрезок [0; 6] для функции f2 не является строго унимодальным, метод квадратичной интерполяции гарантированно находит только локальный, а не обязательно глобальный экстремум. Зависимость количества вычислений от eps показана на

рисунке 7.

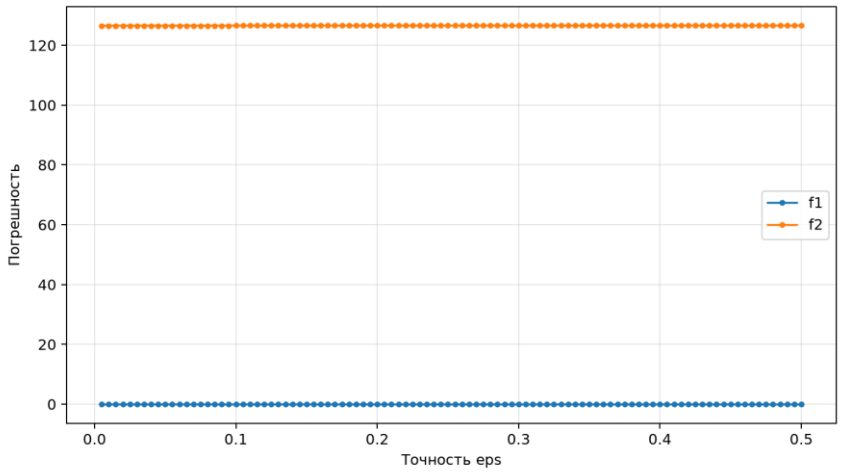


Рисунок 6 – Зависимость погрешности квадратичной интерполяции от eps

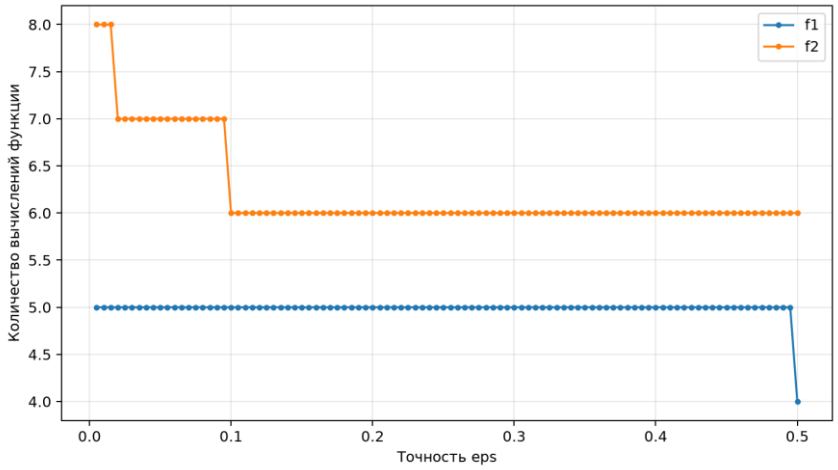


Рисунок 7 – Зависимость числа вычислений квадратичной интерполяции от eps

Число вычислений для обеих функций почти не зависит от eps и остаётся очень маленьким: 4–5 для f1 и 6–8 для f2. Это на два порядка меньше, чем у равномерного поиска, что подтверждает высокую скорость сходимости метода квадратичной интерполяции — при условии, что функция унимодальна на рассматриваемом отрезке.

4.4. Сводная таблица

Итоговые результаты вычислительных экспериментов представлены на рисунке 8. В таблице указаны диапазоны изменения параметров, а также

средние, лучшие и худшие значения погрешности и количества вычислений целевой функции.

Метод	Функция	Параметры	Погрешность средняя	Погреш-ность лучшая	Погрешность худшая	Вычисления средние	Вычисления лучшие	Вычисления худшие
Квадратичная интерполяция	f1	eps = 0.005...0.500, шаг 0.005	0.000e+00	0.000e+00	0.000e+00	5.0	4.0	5.0
Квадратичная интерполяция	f2	eps = 0.005...0.500, шаг 0.005	1.266e+02	1.266e+02	1.266e+02	6.2	6.0	8.0
Равномерный поиск	f1	n = 20...1000, шаг 20	2.983e-03	0.000e+00	4.000e-02	511.0	21.0	1001.0
Равномерный поиск	f2	n = 20...1000, шаг 20	1.505e-02	5.081e-07	3.966e-02	511.0	21.0	1001.0

Рисунок 8 – Сводная таблица результатов вычислительных экспериментов

Из таблицы видно, что метод квадратичной интерполяции требует на два порядка меньше вычислений, чем метод равномерного поиска, и даёт точный результат для унимодальной функции f1. При этом для не вполне унимодальной на заданном отрезке функции f2 он стабильно сходится к локальному, а не глобальному минимуму, тогда как равномерный поиск при достаточно большом n гарантированно находит окрестность глобального минимума ценой значительно большего числа вычислений.

5. Выводы по работе

В ходе выполнения практической работы были реализованы два метода варианта 1: метод равномерного поиска и метод квадратичной интерполяции. Корректность программной реализации проверена на функции из примеров учебника А. В. Пантелеева и Т. А. Летовой: на отрезке [0; 10] оба метода точно нашли $x^* = 3$, $f(x^*) = -18$.

Для функции $f1(x) = (2x + 3)^2 - 8x - 10$, являющейся параболой, метод квадратичной интерполяции оказался идеальным инструментом: он находит точный минимум $x^* = -0{,}5$, $f^* = -2$ за 4–5 вычислений функции независимо от eps, тогда как равномерному поиску для сопоставимой точности требуются сотни вычислений (в среднем 511 при варьировании n от 20 до 1000).

Для функции $f2(x) = 4(x - 5)^2 + e^x(x - 6)^2$, минимум которой лежит вблизи правой границы отрезка [0; 6] ($x^* \approx 5{,}990$, $f^* \approx 3{,}960$), картина иная. Функция не является строго унимодальной на этом отрезке: помимо глобального минимума у неё есть ещё локальный минимум ($x \approx 0{,}668$) и локальный максимум ($x \approx 3{,}917$). Метод квадратичной интерполяции, стартующий с точек 0, 3 и 6, во всех проведённых экспериментах (eps от 0,005 до 0,5) сходится к ближайшему локальному минимуму $x \approx 0{,}668$, давая устойчивую погрешность около 126,55 при очень малом числе вычислений (6–8). Равномерный поиск, напротив, за счёт полного перебора точек отрезка гарантированно находит окрестность истинного глобального минимума, и его погрешность убывает с ростом n почти до нуля (в среднем 0,015 при варьировании n от 20 до 1000), хотя и требует до 1001 вычисления функции.

Таким образом, метод квадратичной интерполяции существенно эффективнее по числу вычислений и даёт точный результат на гладких унимодальных функциях, но ненадёжен на функциях, которые не унимодальны на заданном отрезке, — в этом случае он может сойтись к «неправильному» локальному экстремуму. Метод равномерного поиска менее экономичен, зато

более устойчив: за счёт полного просмотра отрезка он находит окрестность глобального минимума при любой форме функции, если число разбиений n достаточно велико.